



ECOLE
POLYTECHNIQUE
DE BRUXELLES

2025-2026

INFO-H304 : Compléments de programmation et d'algorithmique

Alignement de séquences de protéines avec l'algorithme de Smith-Waterman

Mincapelli Enzo Silva

Makarov Grigorii

Youssef Elias

Jérémie ROLAND

2025

Table des matières

1	Introduction	1
2	Fonctionnement général	2
3	Choix du top 20	4
4	Conception de l'algorithme de Smith-Waterman	5
5	API openMP	6
6	Utilisation de l'intelligence artificielle	7

Dans ce projet, nous avons développé un programme permettant de comparer une séquence de protéine de référence à une base de données de protéines, en utilisant l'algorithme de Smith-Waterman (SW) avec pénalités affines et la matrice de substitution BLOSUM62. Le programme repose sur une lecture directe des fichiers binaires de type BLAST (.pin, .psq, .phr). Pour la remise intermédiaire, il retourne l'identifiant de la protéine de la base dont la séquence est identique à celle de la requête. Pour la remise finale, il retourne les 20 protéines obtenant les meilleurs scores de similarité avec la requête. Le projet comprend une version préliminaire, une version finale et une version finale optimisée. Afin de distinguer les deux versions finales, les classes de la version optimisée sont suffixées par opt. La partie intermédiaire est quant à elle différenciée par le fichier mainprelim.cpp.

Fonctionnement général

Le programme est structuré en plusieurs classes, dont les objets sont tous initialisés dans la fonction main.

Dès l'exécution du programme, un objet de la classe PinfileHandler est créé. Cette classe est chargée de la gestion du fichier .pin. Lors de son initialisation, elle parcourt le fichier .pin afin d'ignorer les informations non pertinentes et d'extraire les données nécessaires à la description de la base de données telles que le nombre total de séquences. Cette étape est essentielle, car elle permet de déterminer précisément les offsets à utiliser pour accéder aux différentes séquences de protéines ainsi que leur identifiant respectif. En effet, les offsets correspondant aux séquences protéiques et aux noms des protéines sont stockés dans le fichier .pin.

Le fichier .psq contient les séquences de protéines encodées sous forme binaire. En effet, chaque acide aminé est représenté par un octet selon une table d'encodage spécifique au format BLAST. Dans le but de convertir ces valeurs binaires en caractères représentant les acides aminés, le programme s'appuie sur une table de correspondance définie dans dict.cpp.

Ensuite, un objet de la classe AlgoSW est instancié. Cette classe est responsable de l'implémentation de l'algorithme SW avec pénalités affines. Elle permet de calculer le score de similarité entre la séquence de requête et les protéines de la base de données.

Dans la fonction main, un objet de la classe SeqfileHandler est également créé. Celui-ci permet de parcourir le fichier .psq, qui contient les séquences de chaque protéine, et, pour la remise finale, de calculer le score d'alignement de chacune d'entre elles avec la séquence de requête. Cette classe renvoie un tableau contenant les 20 meilleurs scores obtenus, ainsi que l'indice correspondant à chaque protéine dans le fichier .psq. Pour la remise intermédiaire, cet objet permet de trouver

l'indice (ordre dans la base de données) de la séquence identique à la séquence requête.

Parallèlement, un objet de la classe PhrfileHandler est également créé, afin de gérer la lecture du fichier .phr. Cette classe permet de récupérer le nom de la protéine associée à une séquence donnée. Grâce à la structure ordonnée des offsets, il est possible, en connaissant l'indice d'une protéine dans la base, de se rendre directement à l'emplacement approprié dans la table des offsets du fichier .phr pour obtenir le nom de la protéine.

Choix du top 20

Au début de l'exécution, le tableau destiné à stocker les meilleurs scores est initialisé avec une taille fixe de 20 et rempli de zéros. Ce tableau est constamment trié par ordre décroissant. Ensuite, au fur et à mesure de la lecture des protéines de la base de données, un score d'alignement est calculé pour chaque protéine à l'aide de l'algorithme SW.

Une fois le score obtenu, le tableau des top 20 est parcouru de la première à la dernière position afin de déterminer l'emplacement correct du nouveau score. Dès qu'un élément strictement inférieur au score à insérer est rencontré, un décalage (shift) de tous les éléments suivants vers la droite est effectué, et le nouveau score est inséré à la position correspondante. Cette procédure garantit que le tableau reste trié après chaque insertion.

Cependant, la gestion du tableau des 20 meilleurs scores, incluant la recherche de la position d'insertion ainsi que le décalage éventuel des éléments, est bornée par une constante. En effet, la complexité dépend uniquement du nombre d'éléments du tableau et non du nombre total de séquences traitées. Puisque la taille du tableau est fixée à 20, ce coût est en $O(1)$ et peut donc être considéré comme négligeable devant celui de l'algorithme SW. Cet algorithme possède en effet une complexité temporelle en $O(m \times n)$, où m et n représentent les longueurs des protéines comparées. C'est pour cette raison que nous n'avons pas utilisé d'autres structures de données pour stocker les 20 protéines ayant le plus grand score.

Conception de l'algorithme de Smith-Waterman

L'algorithme SW est une méthode de programmation dynamique utilisée pour le calcul d'alignements locaux entre deux séquences biologiques. Le fonctionnement de l'algorithme repose sur le calcul d'une matrice de scores, dans laquelle chaque élément correspond au meilleur score d'alignement local se terminant à une position donnée des deux séquences. Les scores sont déterminés à l'aide d'une matrice de substitution, telle que BLOSUM62, et de pénalités (gaps). Afin de garantir le caractère local de l'alignement, les scores négatifs sont remplacés par zéro, ce qui permet de redémarrer un alignement à n'importe quelle position des séquences.

Dans son implémentation classique (telle que donnée dans les références dans le dépôt git du projet), l'algorithme requiert le stockage de trois matrices de dimensions proportionnelles au produit des longueurs des deux séquences comparées. La matrice H stocke le meilleur score d'alignement local se terminant à une position donnée. La matrice E correspond aux alignements se terminant par un gap horizontal et la matrice F correspond aux alignements se terminant par un gap vertical. Cette approche conduit à une complexité mémoire quadratique (complexité en $O(nxm)$).

Cependant, comme seul le score d'alignement local maximal est recherché, il n'est pas indispensable de stocker en mémoire tous les éléments de la matrice. En effet, le calcul du score à une position donnée ne dépend que de la ligne correspondante à cette position (élément à gauche) et de la ligne précédente (élément au-dessus et élément en haut à gauche). Dans le but d'optimiser l'utilisation mémoire, nous pouvons uniquement stocker en mémoire les éléments importants, c'est-à-dire la ligne actuelle et la ligne précédente. La complexité mémoire devient alors $O(n)$, où n correspond à la longueur de la séquence stockée en colonnes.

La version optimisée du projet (exécutable `projetopt`) utilise la bibliothèque de parallélisation OpenMP. Sur les systèmes Linux, OpenMP n'est pas une bibliothèque à installer séparément. L'API OpenMP a été utilisée pour optimiser le code en introduisant une parallélisation établie sur des threads. Cette optimisation constituait la dernière étape du projet : en effet, le temps d'exécution initial atteignait une dizaine de minutes sur la base de données complète, et environ sept secondes sur la base de test. Afin de réduire significativement ce temps, nous avons choisi de paralléliser la lecture et le traitement des protéines.

OpenMP via la directive `"#pragma omp parallel for schedule(dynamic, 64)"` permet de créer plusieurs threads qui travaillent simultanément sur des protéines différentes. Pour répartir efficacement la charge, nous avons utilisé l'option `schedule(dynamic, 64)`. Celle-ci attribue à chaque thread un lot de 64 protéines à traiter, puis lui en donne un nouveau dès qu'il a terminé. Cette stratégie dynamique équilibre mieux le travail entre les threads, car certaines protéines demandent plus de temps de calcul que d'autres. Ainsi, les threads rapides ne restent pas inactifs et la puissance de calcul disponible est exploitée de manière optimale.

Cependant, la parallélisation ne peut pas être appliquée uniformément à toutes les étapes. Lorsqu'un thread souhaite insérer une protéine dans la liste des 20 meilleures, plusieurs threads peuvent entrer en concurrence et tenter de modifier cette zone mémoire en même temps, ce qui risquerait de corrompre les données. Pour éviter cette situation, nous avons utilisé la directive `"#pragma omp critical"`, qui agit comme un mutex. Elle garantit qu'un seul thread à la fois peut accéder à cette section critique du code, assurant ainsi l'intégrité du tableau des meilleures protéines.

Grâce à cette parallélisation par threads, le temps de calcul est réduit à moins d'une seconde pour la base de données de test, et à 5 minutes 30 secondes pour la base de données complète du projet.

Utilisation de l'intelligence artificielle

Dans le cadre de ce travail, l'intelligence artificielle a été utilisée comme outil d'assistance. Elle a permis d'aider au débogage du code, notamment pour l'identification de problèmes liés au parallélisme et à l'utilisation d'OpenMP, ainsi que pour la compréhension et la correction de comportements inattendus du programme.

L'intelligence artificielle a également été employée comme outil d'amélioration de la structuration des idées, ainsi que comme support à la recherche créative, en proposant différentes approches algorithmiques et solutions d'optimisation.

Les décisions finales, la validation des résultats et l'implémentation effective des solutions sont restées sous la responsabilité des auteurs.